Electron-JS 101

A basic guide to a popular JavaScript framework right now

Agnibha Bose | feedback@digit.in

To start with Electron js or simply Electron is an open source library developed by Github for creating cross-platform desktop application with pure HTML, CSS and Javascript. This is accomplished by combining Chromium and NodeJS into a single runtime and apps can be packaged for Mac, Windows and Linux.

Electron started its journey on April, 2013 as the framework on which Github's widely popular and hackable editor Atom was going to be built upon. Both were made open source in the Spring of 2014.

WHAT WE ARE TRYING TO ACHEIVE?

In this article we are trying to build a simple desktop application with the help of Electron and learn some new and cool techniques to create a desktop application from your current web application. Though in this era web-apps are dominating the scene but desktop applications are still widely popular and people tend to use desktop application more than webapps because of their availability and usability.

WHAT ARE THE PREREQUISITES?

This article is NOT for someone who is new to NodeJS. So, before you start, we are assuming that you have a little bit of knowledge about NodeJS. Also a little bit of HTML and CSS knowledge is required to create the frontend part of the project. Here are a couple of links to brush up on NodeJS (https://dgit.in/NodeJS), HTML (https://dgit.in/HTML) and CSS (https://dgit.in/CSS). before we get started with the project.

WHAT WE ARE MAKING TODAY?

Our goal for today is to create a desktop chat messenger. Something sort of like Yahoo Messenger. Where you can register, add friends and chat with them.

THINGS THAT WE ARE GOING TO USE TODAY

We are going to create the FrontEnd part by using HTML and CSS. The Backend will be in NodeJS with the help of WebSockets and A HTTP Server. I am going to use ExpressJS for this purpose. The Github repo link will be shared at the end of the article and you can clone and try something else with the code.

So our basic project structure looks like this.

```
├── client
│   └── package.json
├── README.md
└── server
    └── package.json
```

As you can see in the above image, hhere the client is the client side project which will have all the things that are written for the client, while all the webpages and the server will be on an express server to serve the requests.

So our first goal is to create a server. For that we are going to install a number of pacakges. These packages are shown below.

```
"dependencies": {
  "bcrypt": "^1.0.3",
  "body-parser": "^1.18.2",
  "cookie-parser": "^1.4.3",
  "diskdb": "^0.1.17",
  "express": "^4.16.3",
  "express-ws": "^3.0.0",
  "http-status": "^1.0.1",
  "jsonwebtoken": "^8.2.0"
}
```

Every package has it's own reasons to be in the server. `Bcrypt` is to hash passwords, `body-parser` is to parse request bodies via express, `cookie-parser` is to parse cookies, `diskdb` as a database, `express` as the server, `express-ws` for managing websockets, `http-status` provides handy constants to manage http response code, and finally `jsonwebtoken` for managing auth token.

Now to start we are going to create a webserver at port 8692 that will be serving as our main backend. (code at https://dgit.in/CWSP8692)

Our app will be consisting of 3 main functionalities.

1. Registering an User.
2. Login for an user.
3. Searching and chatting with an user.

So for this purpose we have created a few routes where all of the functionalites lie. Coming to the front end of the application, we've created a single page application for the ease of understanding and also other features are explained in the next section. Now, for the front end we have created a single page index.html. The folder structure for the client looks like this

```
├── node_modules
├── view
├── main.js
├── package.json
└── package-lock.json
```

And within `view` the files and folders are mapped in this way.

```
├── css
│   └── style.css
├── img
│   ├── agent.png
│   ├── anon.png
│   ├── bg.jpg
│   └── nomsg.png
├── index.html
└── js
    └── login.js
```

Now to start with Electron development the package that you need